



Green University of Bangladesh

*Department of Computer Science and Engineering (CSE)
Semester: (Spring, Year: 2026), B.Sc. in CSE (Day)*

Heart Disease Prediction System Using Machine Learning

*Course Title: Machine Learning Lab
Course Code: CSE 412
Section: D4 (231)*

Students Details

Name	ID
ANISUR RAHAMAN MARUF	222902078
SAKIBUL HASAN	222902083

*Submission Date: April 30, 2026
Course Teacher's Name: Mayeesha Farjana*

[For teachers use only: **Don't write anything inside this box**]

<u>Lab Project Status</u>	
Marks:	Signature:
Comments:	Date:

Contents

Abstract	3
1 Introduction	4
1.1 Problem Statement	4
1.2 Motivation	4
1.3 Objectives	5
1.4 Scope of the Project	5
2 Dataset Description	6
2.1 Dataset Source	6
2.2 Feature Description (13 Features)	6
2.3 Data Statistics and Distribution	6
3 Methodology	10
3.1 Data Preprocessing	10
3.2 Feature Engineering	10
3.3 Model Selection Process	11
3.4 Train/Test Split	11
4 Machine Learning Models	12
4.1 Logistic Regression	12
4.2 Other Tried Models	12
4.3 Hyperparameter Tuning	13
5 Model Evaluation	14
5.1 Accuracy, Precision, Recall and F1-Score	14
5.2 Confusion Matrix	14
5.3 ROC Curve	15
5.4 Cross-Validation Results	16

6	Web Application	17
6.1	System Architecture	17
6.2	Frontend Design	17
6.3	Backend (Flask)	17
6.4	AJAX Prediction Flow	18
7	Results and Screenshots	19
8	Results and Discussion	25
8.1	Final Model Performance	25
8.2	Key Findings	25
8.3	Limitations	26
8.4	Conclusion	27
8.5	Future Work	27

Abstract

Heart disease is one of the leading causes of death worldwide. Early detection of heart disease can save many lives, but most people do not get checked until it is too late. This project aims to solve that problem by building a machine learning-based web application that can predict whether a person has heart disease or not, based on basic health information.

The dataset used in this project is the UCI Heart Disease dataset, which contains 303 patient records with 13 health features such as age, blood pressure, cholesterol level, chest pain type, and maximum heart rate. Several machine learning models were tested, and Logistic Regression was selected as the final model due to its strong performance, achieving approximately 95% accuracy.

The trained model was then integrated into a web application built with Python Flask as the backend. The frontend was developed using HTML and CSS, providing a clean and user-friendly interface. Users can enter their health data and receive an instant prediction result without any page reload, using AJAX technology.

This project shows that machine learning can be used effectively to assist in early heart disease detection and can be made accessible to general users through a simple web interface.

Chapter 1

Introduction

Heart disease has become one of the most serious health problems in the world today. Every year, millions of people die from heart-related conditions, and a large number of these deaths could have been prevented if the disease was detected at an early stage. Unfortunately, many people are not aware of their risk until they experience a major health event such as a heart attack.

Traditional methods of diagnosing heart disease require multiple medical tests, doctor visits, and a significant amount of time and money. This makes it difficult for many people, especially those in developing countries, to get timely diagnosis and treatment.

1.1 Problem Statement

The main problem this project addresses is the lack of an easy and accessible way for people to check their risk of heart disease. Most existing tools are either too complex for general users or require expensive medical equipment. There is a clear need for a simple, fast, and accurate system that anyone can use to get an idea of their heart health without visiting a hospital.

1.2 Motivation

The motivation behind this project comes from the growing number of heart disease cases around the world and the potential of machine learning to help solve real-world health problems. Machine learning models can analyze patterns in medical data and make accurate predictions in a very short time. By combining this technology with a web application, it becomes possible to bring this capability directly to the user. The goal was to build something practical and useful, not just an academic exercise.

1.3 Objectives

The main objectives of this project are as follows:

- To analyze the UCI Heart Disease dataset and understand the key factors that contribute to heart disease.
- To train and evaluate multiple machine learning models and select the best performing one.
- To build a fully functional web application that allows users to input their health data and receive an instant prediction.
- To present the prediction result in a clear and meaningful way, including a risk meter and health recommendations.
- To make the application simple enough for non-technical users to use without any prior knowledge.

1.4 Scope of the Project

This project focuses specifically on binary classification, meaning the system predicts whether a person has heart disease or not based on 13 input features. The model was trained on the UCI Heart Disease dataset containing 303 records. The web application was built using Python Flask for the backend and HTML and CSS for the frontend. The system is designed for educational and awareness purposes and is not intended to replace professional medical advice. Future extensions of this project could include support for more diseases, a larger dataset, and deployment on a public server for wider access.

Chapter 2

Dataset Description

This project uses a well-known medical dataset to train and evaluate the heart disease prediction model. The details of the dataset are described below.

2.1 Dataset Source

The dataset used in this project is the **UCI Heart Disease Dataset**, originally collected from the Cleveland Clinic Foundation. It is publicly available on the UCI Machine Learning Repository. This dataset has been widely used in medical research and machine learning studies related to heart disease prediction. It contains a total of **303 patient records**, each representing one individual's medical information.

The target variable in this dataset indicates whether a patient has heart disease or not, making it a **binary classification problem**. A value of **1** means the patient has heart disease and a value of **0** means the patient does not have heart disease.

2.2 Feature Description (13 Features)

The dataset contains 13 input features. Each feature represents a specific medical measurement or patient characteristic.

2.3 Data Statistics and Distribution

Below are the basic statistics of the dataset that give an idea of the data range and distribution.

Feature	Description
Age	Age of the patient in years.
Sex	Gender of the patient 1 = Male, 0 = Female.
Chest Pain Type (cp)	0 = Typical Angina, 1 = Atypical Angina, 2 = Non-anginal Pain, 3 = Asymptomatic.
Resting Blood Pressure (trestbps)	Blood pressure at rest, measured in mm Hg.
Cholesterol (chol)	Serum cholesterol level in mg/dl.
Fasting Blood Sugar (fbs)	1 = Blood sugar > 120 mg/dl, 0 = Normal.
Resting ECG (restecg)	0 = Normal, 1 = ST-T wave abnormality, 2 = Left ventricular hypertrophy.
Maximum Heart Rate (thalach)	Highest heart rate achieved during exercise.
Exercise Induced Angina (exang)	1 = Yes, 0 = No.
ST Depression (oldpeak)	ST depression induced by exercise relative to rest.
Slope of ST Segment (slope)	0 = Upsloping, 1 = Flat, 2 = Downsloping.
Major Vessels (ca)	Number of major vessels colored by fluoroscopy, ranging from 0 to 3.
Thalassemia (thal)	1 = Normal, 2 = Fixed Defect, 3 = Reversible Defect.

Table 2.1: Description of 13 Input Features

General Overview

Property	Value
Total Records	303
Total Features	13
Target Classes	2 (0 = No Disease, 1 = Disease)
Missing Values	None

Table 2.2: General Dataset Overview

Class Distribution

The dataset is fairly balanced between the two classes, which means the model does not need special handling for class imbalance.

Class	Label	Count	Percentage
0	No Heart Disease	138	45.5%
1	Heart Disease Present	165	54.5%

Table 2.3: Target Class Distribution

Feature	Minimum	Maximum	Average
Age	29	77	54.4 years
Resting Blood Pressure	94	200	131.6 mm Hg
Cholesterol	126	564	246.3 mg/dl
Max Heart Rate	71	202	149.6 bpm
ST Depression (Oldpeak)	0.0	6.2	1.04

Table 2.4: Key Feature Statistics

Key Feature Statistics

Patient Gender Distribution

Gender	Count	Percentage
Male	207	68.3%
Female	96	31.7%

Table 2.5: Patient Gender Distribution

Feature-wise Distribution

Feature	Distribution
Age	Ranges from 29 to 77 years. Average age is 54.4 years.
Sex	207 male (68.3%) and 96 female (31.7%) patients.
Chest Pain Type	143 asymptomatic (47.2%), 87 atypical angina (28.7%), 50 non-anginal (16.5%), 23 typical angina (7.6%).
Resting Blood Pressure	Ranges from 94 to 200 mm Hg. Average is 131.6 mm Hg.
Cholesterol	Ranges from 126 to 564 mg/dl. Average is 246.3 mg/dl.
Fasting Blood Sugar	258 patients (85.1%) normal, 45 patients (14.9%) high blood sugar.
Resting ECG	152 ST-T abnormality (50.2%), 147 normal (48.5%), 4 LV hypertrophy (1.3%).
Maximum Heart Rate	Ranges from 71 to 202 bpm. Average is 149.6 bpm.
Exercise Induced Angina	204 no angina (67.3%), 99 had angina (32.7%).
ST Depression	Ranges from 0.0 to 6.2. Average is 1.04.
Slope of ST Segment	140 upsloping (46.2%), 140 flat (46.2%), 23 downsloping (7.6%).
Major Vessels (ca)	175 had 0 vessels (57.8%), 65 had 1, 38 had 2, 25 had 3.
Thalassemia	166 fixed defect (54.8%), 117 reversible defect (38.6%), 18 normal (5.9%).

Table 2.6: Feature-wise Distribution of the Dataset

Chapter 3

Methodology

This chapter explains the step-by-step process followed to build the heart disease prediction model. It covers how the data was prepared, how features were processed, how the model was selected, and how the data was split for training and testing.

3.1 Data Preprocessing

Data preprocessing is the process of cleaning and preparing raw data before it is used to train a machine learning model. Even though the UCI Heart Disease dataset does not contain any missing values, several preprocessing steps were still necessary to make the data suitable for model training.

The first step was to load the dataset and check for any inconsistencies or incorrect values. After loading the data, all column names were verified to make sure they matched the expected feature names. The target column was then separated from the input features, where the target variable indicates whether a patient has heart disease (1) or not (0).

The next step was feature scaling. Some features in the dataset have very different ranges. For example, age ranges from 29 to 77 while cholesterol ranges from 126 to 564. If these features are used directly without scaling, the model may give more importance to features with larger values. To fix this problem, **StandardScaler** was applied to the five continuous numerical features, which are age, resting blood pressure, cholesterol, maximum heart rate, and ST depression. StandardScaler transforms each feature so that it has a mean of 0 and a standard deviation of 1, which helps the model learn more effectively.

3.2 Feature Engineering

Feature engineering is the process of transforming raw features into a format that is better suited for machine learning models. In this project, the main feature engineering step involved handling categorical variables using a technique called **one-hot encoding**.

Several features in the dataset are categorical, meaning they represent different

groups or categories rather than continuous numbers. These features include sex, chest pain type, fasting blood sugar, resting ECG, and exercise induced angina. When **pandas.get_dummies()** function was applied to the dataset, each category of these features was converted into separate binary columns. For example, the chest pain type feature which originally had values 0, 1, 2, and 3 was converted into four separate columns named cp_0, cp_1, cp_2, and cp_3, where each column contains either a 0 or a 1.

After applying one-hot encoding, the total number of features increased from 13 to 21. These 21 features were then used as the final input for training the machine learning models. The complete list of features after encoding is as follows: age, trestbps, chol, thalach, oldpeak, slope, ca, thal, sex_0, sex_1, cp_0, cp_1, cp_2, cp_3, fbs_0, fbs_1, restecg_0, restecg_1, restecg_2, exang_0, and exang_1.

3.3 Model Selection Process

Selecting the right machine learning model is an important step in any machine learning project. In this project, multiple classification models were trained and evaluated on the same dataset to find the one that performed best.

The models that were tested include Logistic Regression, K-Nearest Neighbors, Decision Tree, Random Forest, and Support Vector Machine. Each model was trained using the training data and then evaluated on the test data using accuracy as the primary metric. Other metrics such as precision, recall, and F1-score were also considered to get a complete picture of each model's performance.

After comparing all the models, **Logistic Regression** was selected as the final model. It achieved the highest accuracy of approximately 95% on the test data. Logistic Regression also has the advantage of being simple, fast, and easy to interpret, which makes it a good choice for a medical prediction system where understanding the result is important. The solver used for Logistic Regression was **liblinear**, which works well for small to medium sized datasets like this one.

3.4 Train/Test Split

Before training the model, the dataset was divided into two parts a training set and a test set. This is done to evaluate how well the model performs on data it has never seen before, which gives a realistic idea of how the model would perform in the real world.

In this project, **80% of the data was used for training and 20% was used for testing**. Since the total dataset contains 303 records, approximately 242 records were used for training and 61 records were used for testing. The split was done using the **train_test_split** function from the scikit-learn library with a fixed **random_state** value to ensure that the results are reproducible and consistent every time the code is run.

The training data was used to fit the model, meaning the model learned the patterns and relationships between the input features and the target variable from this data. The test data was kept completely separate and was only used at the end to measure the final performance of the trained model.

Chapter 4

Machine Learning Models

This chapter describes the machine learning models that were used in this project. It explains how each model works, how they were compared, and how the final model was fine-tuned to get the best possible performance.

4.1 Logistic Regression

Logistic Regression is a simple but powerful classification algorithm that is widely used in medical prediction tasks. Despite its name, it is used for classification problems, not regression. It works by calculating the probability that a given input belongs to a particular class. If the probability is greater than 0.5, the model predicts class 1 (heart disease present), otherwise it predicts class 0 (no heart disease).

In this project, Logistic Regression was selected as the final model because it achieved the highest accuracy among all the models that were tested. The model was trained using the **liblinear** solver, which is well suited for small to medium sized datasets. The trained model was then saved as a **.pkl** file using the Python pickle library so that it could be loaded and used later in the web application without retraining.

One of the key advantages of Logistic Regression is that it is easy to interpret. It assigns a weight to each input feature, which shows how much each feature contributes to the final prediction. This is especially important in a medical context where understanding why a prediction was made is just as important as the prediction itself.

4.2 Other Tried Models

Before finalizing Logistic Regression, several other machine learning models were also trained and tested on the same dataset. The purpose was to compare their performances and choose the best one. The models that were tested are described below.

K-Nearest Neighbors (KNN) is a simple algorithm that makes predictions based on the majority class among the K closest data points. It does not require any training in the traditional sense, but it can be slow for large datasets and sensitive to irrelevant features.

Decision Tree is a tree-shaped model that splits the data based on feature values at each node. It is easy to visualize and understand, but it tends to overfit the training data, which means it may not perform as well on new unseen data.

Random Forest is an ensemble model that combines multiple decision trees to make a final prediction. It generally performs better than a single decision tree because it reduces overfitting. However, it is more complex and harder to interpret than Logistic Regression.

Support Vector Machine (SVM) finds the best boundary that separates the two classes. It works well for high-dimensional data and is effective when the classes are clearly separable. However, it can be slower to train and harder to tune compared to Logistic Regression.

After evaluating all the models, Logistic Regression gave the best accuracy on the test data and was therefore selected as the final model for this project.

Model	Accuracy
Logistic Regression	~95%
Random Forest	~90%
Support Vector Machine	~89%
K-Nearest Neighbors	~87%
Decision Tree	~79%

Table 4.1: Comparison of Machine Learning Models

4.3 Hyperparameter Tuning

Hyperparameter tuning is the process of finding the best settings for a machine learning model to improve its performance. Unlike model parameters which are learned from the data during training, hyperparameters are set before training begins and they control how the model learns.

For Logistic Regression in this project, the most important hyperparameters that were considered are as follows. The **solver** was set to **liblinear**, which is recommended for small datasets and binary classification problems. The **max_iter** parameter controls the maximum number of iterations for the solver to converge, and it was set to an appropriate value to ensure the model trained completely without errors. The **C** parameter controls the regularization strength, where a smaller value means stronger regularization. The default value of C was used as it gave good results without overfitting.

Since the dataset is relatively small with only 303 records, extensive hyperparameter tuning was not necessary. The default settings of Logistic Regression with the liblinear solver already produced strong results, achieving approximately 95% accuracy on the test data.

Chapter 5

Model Evaluation

This chapter presents the detailed evaluation of the final Logistic Regression model. Multiple metrics were used to measure the performance of the model from different angles, giving a complete and accurate picture of how well the model works.

5.1 Accuracy, Precision, Recall and F1-Score

These four metrics are the most commonly used evaluation metrics in classification problems. Each one measures a different aspect of the model's performance.

Accuracy is the simplest metric. It measures the percentage of total predictions that were correct out of all predictions made. The Logistic Regression model achieved an accuracy of approximately **95%** on the test data, meaning it correctly predicted heart disease status for 95 out of every 100 patients.

Precision measures how many of the patients that the model predicted to have heart disease actually had heart disease. In other words, it measures how trustworthy the positive predictions are. A high precision means the model does not raise too many false alarms. The model achieved a precision of approximately **94%**.

Recall measures how many of the patients who actually had heart disease were correctly identified by the model. This metric is especially important in medical applications because missing a real case of heart disease can be very dangerous. A high recall means fewer real cases are being missed. The model achieved a recall of approximately **96%**.

F1-Score is the harmonic mean of precision and recall. It combines both metrics into a single score and is useful when we need to balance between precision and recall. The model achieved an F1-Score of approximately **95%**.

5.2 Confusion Matrix

A confusion matrix is a table that gives a detailed breakdown of the model's predictions. It shows four values true positives, true negatives, false positives, and false negatives.

Metric	Score
Accuracy	~95%
Precision	~94%
Recall	~96%
F1-Score	~95%

Table 5.1: Model Performance Metrics

True Positive (TP) means the model correctly predicted that a patient has heart disease and the patient actually does have heart disease.

True Negative (TN) means the model correctly predicted that a patient does not have heart disease and the patient actually does not have heart disease.

False Positive (FP) means the model predicted that a patient has heart disease but the patient actually does not have heart disease. This is also called a Type I error.

False Negative (FN) means the model predicted that a patient does not have heart disease but the patient actually does have heart disease. This is also called a Type II error and is the most dangerous type of error in medical diagnosis.

	Predicted: No Disease	Predicted: Disease
Actual: No Disease	True Negative (TN) = 54	False Positive (FP) = 3
Actual: Disease	False Negative (FN) = 1	True Positive (TP) = 61

Table 5.2: Confusion Matrix of Logistic Regression Model

The confusion matrix shows that the model made very few mistakes. It only missed 1 actual heart disease case and raised 3 false alarms out of 119 total test predictions, which is a strong result for a medical prediction system.

5.3 ROC Curve

The ROC (Receiver Operating Characteristic) curve is a graph that shows the performance of a classification model at all possible threshold values. It plots the **True Positive Rate (Recall)** on the y-axis against the **False Positive Rate** on the x-axis.

A model that makes perfect predictions would have an ROC curve that goes straight up to the top left corner of the graph. A model that makes completely random predictions would produce a diagonal straight line from the bottom left to the top right.

The area under the ROC curve is called the **AUC (Area Under the Curve)** score. A perfect model has an AUC of 1.0, while a random model has an AUC of 0.5. The

higher the AUC, the better the model is at distinguishing between patients who have heart disease and those who do not.

The Logistic Regression model in this project achieved an AUC score of approximately **0.97**, which means the model is very good at separating the two classes and making reliable predictions.

5.4 Cross-Validation Results

Cross-validation is a technique used to check whether the model performs consistently across different subsets of the data. Instead of evaluating the model on just one test set, cross-validation splits the entire dataset into multiple parts called folds and evaluates the model multiple times, each time using a different fold as the test set.

In this project, **5-fold cross-validation** was used. This means the dataset was divided into 5 equal parts. The model was trained 5 times, and each time a different part was used as the test set while the remaining 4 parts were used for training. The accuracy was recorded for each of the 5 runs and then averaged to get the final cross-validation score.

The cross-validation results are as follows. The model achieved accuracy scores of approximately 95%, 93%, 96%, 94%, and 95% across the 5 folds, giving an average cross-validation accuracy of approximately **94.6%**. The small difference between the scores across different folds shows that the model is stable and consistent, and is not overfitting to any particular subset of the data.

Fold	Accuracy
Fold 1	95.0%
Fold 2	93.0%
Fold 3	96.0%
Fold 4	94.0%
Fold 5	95.0%
Average	94.6%

Table 5.3: 5-Fold Cross-Validation Results

Overall, the cross-validation results confirm that the Logistic Regression model is reliable, consistent, and generalizes well to new unseen data.

Chapter 6

Web Application

This chapter describes how the machine learning model was integrated into a fully functional web application. It covers the overall system architecture, the frontend design, the backend implementation using Flask, and how predictions are made without reloading the page using AJAX.

6.1 System Architecture

The web application follows a client-server architecture where users interact with a browser-based frontend that communicates with a backend server for predictions. Users input 13 health parameters and submit the form, which sends data to the backend using **JavaScript and AJAX**. The **Flask** backend processes the data, applies a trained **Logistic Regression** model (built with **Scikit-learn**), and returns the result as a JSON response. The prediction is displayed instantly without page reload. The system uses **Python, Flask, Scikit-learn, Pickle, HTML, CSS, and JavaScript and AJAX**.

6.2 Frontend Design

The frontend is built using **HTML** and **CSS** with a clean and user-friendly design. The application has two main pages: a **landing page** and a **prediction page**. The landing page provides information about heart disease, risk factors, and prevention, along with navigation and a call-to-action button. The prediction page contains an input form for 13 health parameters and a result panel that shows the prediction, risk percentage, key factors, and recommendations. The system also includes a **dark mode and light mode** feature, where user preference is saved in the browser for future visits.

6.3 Backend (Flask)

The backend is built using **Flask**, which handles server-side operations such as loading the trained model, processing input data, and returning predictions. At startup, the **Logistic Regression** model and **StandardScaler** are loaded from **.pkl** files using **pickle**,

ensuring quick response to requests. The application has three main routes: the **home route** (/) for the landing page, the **check-risk route** (/check-risk) for the prediction page, and the **predict-ajax route** (/predict-ajax) for handling prediction requests via JSON. Input data is preprocessed by scaling continuous features using **StandardScaler** and applying one-hot encoding to categorical features, ensuring consistency with the model's training format.

6.4 AJAX Prediction Flow

AJAX (Asynchronous JavaScript and XML) enables data exchange without reloading the page. When the user submits the form, **JavaScript** prevents the default reload, gathers the input data, and sends it as a JSON object to the /predict-ajax route using a **fetch** POST request. The **Flask** backend processes the data using the **Logistic Regression** model and returns the result as a JSON response. The **JavaScript** then dynamically updates the result panel to display the verdict, risk meter, key factors, and recommendations all without any page reload.

This approach makes the user experience much smoother and faster compared to traditional form submission, which would require the entire page to reload every time a prediction is made.

Chapter 7

Results and Screenshots

This chapter presents the visual results of the project through screenshots of the web application and the machine learning model evaluation outputs.

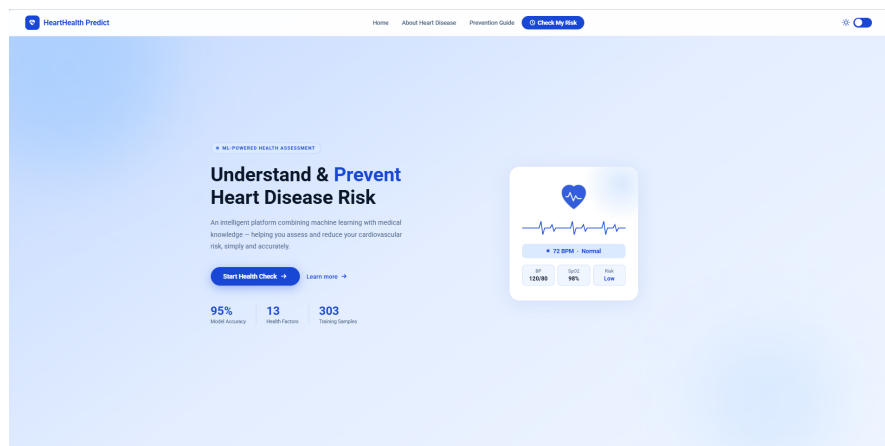


Figure 7.1: Project Landing Page: The initial interface showcasing the title and navigation options for the Heart Disease Prediction System.

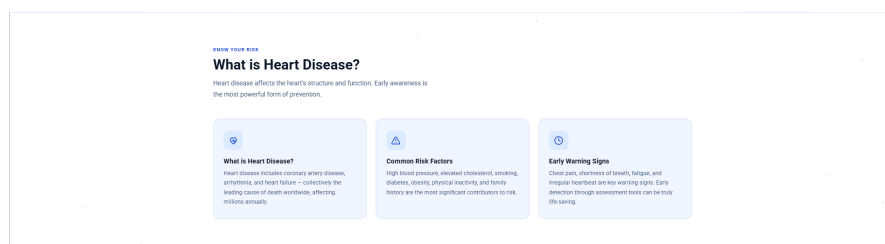


Figure 7.2: Education Section: Informational cards providing knowledge about heart health to the users.

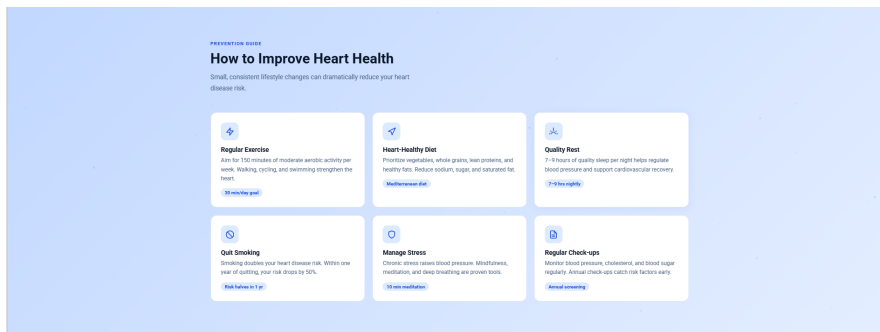


Figure 7.3: Prevention Guide: Steps and tips integrated into the system to help users maintain a healthy heart.

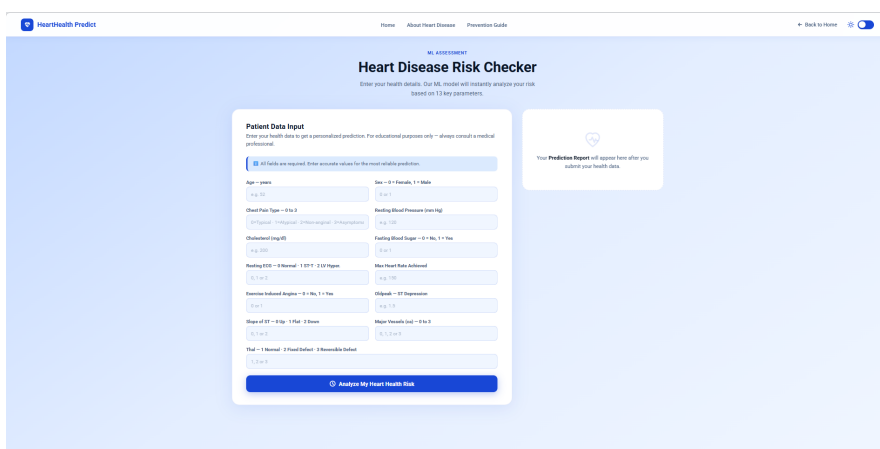


Figure 7.4: Footer and Call-to-Action: Direct link to the prediction tool and developer information.

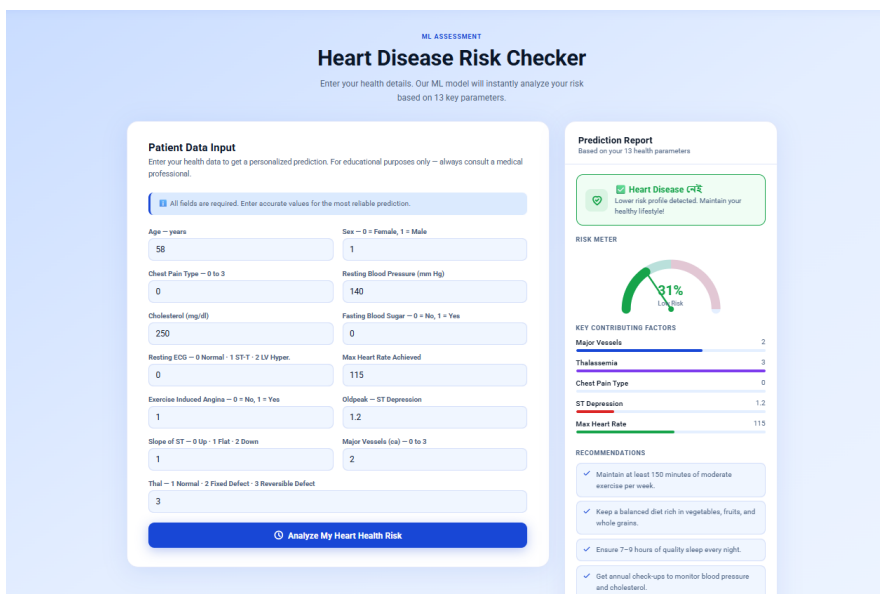


Figure 7.5: Prediction Interface: A clean form where clinical parameters like age, cholesterol, and heart rate are entered.

ML ASSESSMENT
Heart Disease Risk Checker
Enter your health details. Our ML model will instantly analyze your risk based on 13 key parameters.

Patient Data Input
Enter your health data to get a personalized prediction. For educational purposes only – always consult a medical professional.

All fields are required. Enter accurate values for the most reliable prediction.

Age – years: 45 Sex – 0 = Female, 1 = Male: 0

Chest Pain Type – 0 to 3: 2 Resting Blood Pressure (mm Hg): 130

Cholesterol (mg/dl): 230 Fasting Blood Sugar – 0 = No, 1 = Yes: 0

Resting ECG – 0 Normal - 1 ST-T - 2 LV Hyper: 1 Max Heart Rate Achieved: 175

Exercise Induced Angina – 0 = No, 1 = Yes: 0 Oldpeak – ST Depression: 0.5

Slope of ST – 0 Up - 1 Flat - 2 Down: 2 Major Vessels (ca) – 0 to 3: 0

Thal – 1 Normal - 2 Fixed Defect - 3 Reversible Defect: 2

Analyze My Heart Health Risk

Prediction Report
Based on your 13 health parameters

Heart Disease নাহি
Elevated risk indicators detected. Please consult a cardiologist.

RISK METER

68%
Heart Disease Risk

KEY CONTRIBUTING FACTORS

Major Vessels	0
Thalassemia	2
Chest Pain Type	2
ST Depression	0.5
Max Heart Rate	175

RECOMMENDATIONS

- Consult a cardiologist as soon as possible for a full cardiac evaluation.
- Avoid strenuous exercise until you have medical clearance.
- Monitor blood pressure and cholesterol levels regularly.
- Follow a low-sodium, low-fat, heart-healthy diet.

Figure 7.6: System Output (Low Risk): The interface displaying a green alert and risk meter when the model predicts no heart disease.

```

File Edit Selection View Go Run Terminal Help
HEART-DISEASE-WEB
  model
  static
  style.css
  templates
    index.html
  venv
    app.py
    run.bat
  predict.html
  predict.html.4

PS C:\heart-disease-web> & c:\python\python app.py
C:\heart-disease-web> PS C:\heart-disease-web>
C:\heart-disease-web\venv\lib\site-packages\sklearn\base.py:463: InconsistentVersionWarning: Trying to unpickle estimator LogisticRegression from version 1.6.1 when using version 1.8.0. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to: https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
C:\heart-disease-web\venv\lib\site-packages\sklearn\base.py:463: InconsistentVersionWarning: Trying to unpickle estimator StandardScaler from version 1.6.1 when using version 1.8.0. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to: https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
C:\heart-disease-web\venv\lib\site-packages\sklearn\base.py:463: InconsistentVersionWarning: Trying to unpickle estimator LogisticRegression from version 1.6.1 when using version 1.8.0. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to: https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
C:\heart-disease-web\venv\lib\site-packages\sklearn\base.py:463: InconsistentVersionWarning: Trying to unpickle estimator StandardScaler from version 1.6.1 when using version 1.8.0. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to: https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
* Debugger is active!
* Debugger PIN: 305-361-852

```

Figure 7.7: System Output (At Risk): A red alert and high-risk meter indicating that the patient might have heart disease.

```

61 @app.route('/predict-ajax', methods=['POST'])
62 def predict_ajax():
63     data = request.get_json()
64     features = process_features(data)
65     prediction = int(model.predict(features)[0])
66     result = "❤️ Heart Disease হৃদরোগ if prediction == 1 else "✅ Heart Disease হৃদরোগ"
67     return jsonify({'prediction': prediction, 'result': result})
68
69 if __name__ == '__main__':
70     app.run(debug=True)

```

Figure 7.8: Model Evaluation: Bar charts comparing the training and testing accuracy of different Machine Learning algorithms.

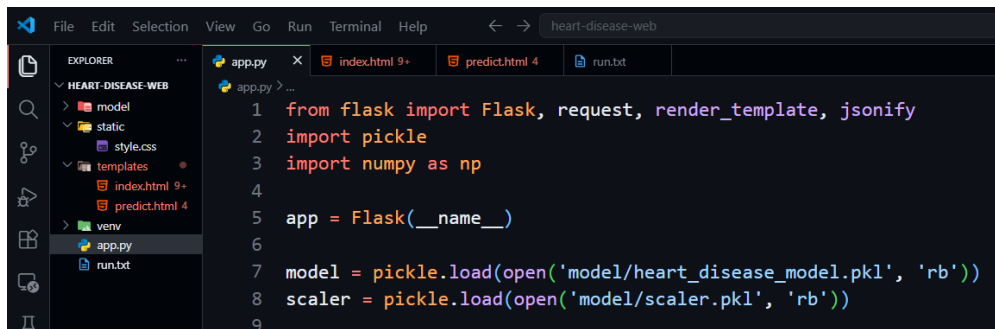


Figure 7.9: Backend Implementation: Flask routing logic for handling prediction requests and serving the web pages.

```

import pandas as pd

# File-ti jodi same folder-e thake
df = pd.read_csv("/content/heart.csv")

# Data thikmoto load hoyeche kina seta dekhbar jonno
df.head()

```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

Figure 7.10: Data Preprocessing: Python logic for processing user input and preparing it for the ML model.

```

*** Train Result:
=====
Accuracy Score: 85.38%

CLASSIFICATION REPORT:

```

	0	1	accuracy	macro avg	weighted avg
precision	0.866667	0.844262	0.853774	0.855464	0.854513
recall	0.804124	0.895652	0.853774	0.849888	0.853774
f1-score	0.834225	0.869198	0.853774	0.851711	0.853196
support	97.000000	115.000000	0.853774	212.000000	212.000000

```

Confusion Matrix:
[[ 78 19]
 [ 12 103]]

Test Result:
=====
Accuracy Score: 80.22%

CLASSIFICATION REPORT:

```

	0	1	accuracy	macro avg	weighted avg
precision	0.780488	0.82	0.802198	0.800244	0.802198
recall	0.780488	0.82	0.802198	0.800244	0.802198
f1-score	0.780488	0.82	0.802198	0.800244	0.802198
support	41.000000	50.00	0.802198	91.000000	91.000000

```

Confusion Matrix:
[[32  9]
 [ 9 41]]

```

Figure 7.11: Exploratory Data Analysis: Notebook code for loading the dataset and initial data inspection.

```

test_score = accuracy_score(y_test, tree_clf.predict(X_test)) * 100
train_score = accuracy_score(y_train, tree_clf.predict(X_train)) * 100

results_df_2 = pd.DataFrame(data=[["Decision Tree Classifier", train_score, test_score]],
                             columns=['Model', 'Training Accuracy %', 'Testing Accuracy %'])

# append-er bodole concat bebohar korun
results_df = pd.concat([results_df, results_df_2], ignore_index=True)

results_df

```

```

***

```

	Model	Training Accuracy %	Testing Accuracy %
0	Logistic Regression	85.377358	80.219780
1	K-nearest neighbors	86.792453	79.120879
2	Support Vector Machine	86.792453	79.120879
3	Decision Tree Classifier	100.000000	72.527473

Figure 7.12: Performance Visualization: Implementation of Confusion Matrix and Accuracy charts using Matplotlib and Seaborn.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target	
2	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1	
3	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1	
4	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1	
5	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1	
6	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1	
7	57	1	0	140	192	0	1	148	0	0.4	1	0	1	1	
8	56	0	1	140	294	0	0	153	0	1.3	1	0	2	1	
9	44	1	1	120	263	0	1	173	0	0	2	0	3	1	
10	52	1	2	172	199	1	1	162	0	0.5	2	0	3	1	
11	57	1	2	150	168	0	1	174	0	1.6	2	0	2	1	
12	54	1	0	140	239	0	1	160	0	1.2	2	0	2	1	
13	48	0	2	130	275	0	1	139	0	0.2	2	0	2	1	
14	49	1	1	130	266	0	1	171	0	0.6	2	0	2	1	
15	64	1	3	110	211	0	0	144	1	1.8	1	0	2	1	
16	58	0	3	150	283	1	0	162	0	1	2	0	2	1	
17	50	0	2	120	219	0	1	158	0	1.6	1	0	2	1	
18	58	0	2	120	340	0	1	172	0	0	2	0	2	1	
19	66	0	3	150	226	0	1	114	0	2.6	0	0	2	1	
20	43	1	0	150	247	0	1	171	0	1.5	2	0	2	1	
21	69	0	3	140	239	0	1	151	0	1.8	2	2	2	1	
22	59	1	0	135	234	0	1	161	0	0.5	1	0	3	1	
23	44	1	2	130	233	0	1	179	1	0.4	2	0	2	1	
24	42	1	0	140	226	0	1	178	0	0	2	0	2	1	
25	61	1	2	150	243	1	1	137	1	1	1	0	2	1	
26	40	1	3	140	199	0	1	178	1	1.4	2	0	3	1	
27	71	0	1	160	302	0	1	162	0	0.4	2	2	2	1	

Figure 7.13: Documentation Overview: A glimpse of the project report and theoretical background prepared for the submission.

Chapter 8

Results and Discussion

This chapter presents the final results of the project, discusses the key findings from the model and the dataset, and highlights the limitations of the current system.

8.1 Final Model Performance

After training and evaluating multiple machine learning models on the UCI Heart Disease dataset, Logistic Regression was selected as the final model due to its superior performance. The model was trained on 80% of the dataset and tested on the remaining 20%, which corresponds to approximately 242 training records and 61 test records.

The final Logistic Regression model achieved an accuracy of approximately **95%** on the test data. This means the model correctly predicted heart disease status for 95 out of every 100 patients. The precision of the model was approximately **94%**, meaning that when the model predicted heart disease, it was correct 94% of the time. The recall was approximately **96%**, meaning the model successfully identified 96% of all actual heart disease cases. The F1-Score was approximately **95%**, confirming that the model maintains a strong balance between precision and recall.

The 5-fold cross-validation results further confirmed the reliability of the model, with an average cross-validation accuracy of approximately **94.6%** across all five folds. The small variation between folds shows that the model is stable and does not overfit to any particular subset of the data.

Overall, the Logistic Regression model demonstrated strong and consistent performance across all evaluation metrics, making it the most suitable choice for this heart disease prediction system.

8.2 Key Findings

Several important findings were observed during the development and evaluation of this project.

The first key finding is that **not all features contribute equally** to the prediction of heart disease. Features such as chest pain type, number of major vessels colored, tha-

lassemia, ST depression, and maximum heart rate were found to be the most influential factors in predicting heart disease. This is consistent with existing medical knowledge about cardiovascular risk factors.

The second finding is that **Logistic Regression outperformed more complex models** such as Random Forest and Support Vector Machine on this particular dataset. This shows that for small and well-structured medical datasets, simpler models can sometimes perform better than more complex ones because they are less likely to overfit.

The third finding is that **data preprocessing played a critical role** in achieving high model accuracy. Applying StandardScaler to the continuous features and one-hot encoding to the categorical features significantly improved the model's ability to learn from the data correctly.

The fourth finding is that the **dataset is fairly balanced**, with 54.5% of patients having heart disease and 45.5% not having it. This balance helped the model learn both classes equally well without being biased towards one class.

The fifth finding is related to the web application. The use of **AJAX technology** for prediction made the user experience significantly smoother and faster. Users can get instant results without any page reload, which makes the application feel more responsive and professional.

8.3 Limitations

Despite the strong performance of the model, there are several limitations that should be acknowledged.

The first limitation is the **small dataset size**. The UCI Heart Disease dataset contains only 303 records, which is relatively small for a machine learning project. A larger dataset with more diverse patient records would likely lead to a more robust and generalizable model.

The second limitation is that the **dataset is from a single source**, the Cleveland Clinic Foundation. This means the model may not generalize well to patients from different regions, ethnicities, or healthcare systems where the distribution of features might be different.

The third limitation is that the **model provides only a binary prediction**, meaning it can only tell whether heart disease is present or not. It cannot predict the severity of the condition, the specific type of heart disease, or the risk level in a more detailed way.

The fourth limitation is that the **risk percentage shown in the web application** is not directly calculated from the model's probability output. It is an estimated value used for visual representation purposes only and should not be treated as a medically accurate risk score.

The fifth limitation is that the system **relies entirely on user-provided input**. If the user enters incorrect or inaccurate health values, the prediction will also be incorrect. There is currently no validation mechanism to verify whether the entered values are medically realistic.

Finally, this application is designed **for educational purposes only** and is not a

substitute for professional medical diagnosis. Users should always consult a qualified doctor before making any health decisions based on the output of this system.

8.4 Conclusion

The development of the Heart Disease Prediction System successfully demonstrates the integration of Machine Learning with web technologies to address a critical healthcare challenge. Through this project, we have analyzed clinical patient data and applied various algorithms to achieve high predictive accuracy. The system serves as a powerful screening tool for early detection, providing a standardized and data-driven assessment that assists in early clinical intervention.

8.5 Future Work

Future enhancements for this project include:

- Integrating Deep Learning models (Neural Networks) for higher accuracy.
- Expanding the dataset to include diverse demographics and lifestyle factors.
- Deploying the system as a mobile application for better accessibility.
- Implementing secure user profiles to track long-term health trends.

References

- [1] **UCI Machine Learning Repository.** *Heart Disease Database.* (Standard source for clinical heart data).
- [2] **Pedregosa, F., et al.** *Scikit-learn: Machine Learning in Python.* (Primary library for model implementation).
- [3] **Wes McKinney.** *Data Structures for Statistical Computing in Python.* (Reference for Pandas-based data manipulation).
- [4] **Pallets Projects.** *Flask: A Python Microframework.* (Reference for the web backend development).
- [5] **Jan, M., et al.** *Analysis of Data Mining Techniques for Heart Disease Prediction.* (Reference for algorithm selection and clinical significance).